Project: Ted Talk Views Prediction

Project Type - EDA/Regression/Supervised

Contribution - Individual

Project Summary -

Project Summary: TED Talk View Prediction

This project aims to predict the number of views a TED Talk video is likely to receive based on various features extracted from the TED Talk dataset. TED Talks, known for their impactful and inspiring content, span a wide variety of topics and speakers. By analyzing TED Talk data, this project explores the relationship between various attributes of TED Talks and their viewership. Understanding this relationship can help content creators, event organizers, and marketers in crafting more engaging and widely viewed talks.

dataset link

click here (https://drive.google.com/file/d/1qT8kO-wsUD6hRMDZjjFaGVKZlxUx3MBD/view?usp=sharing) to access dataset

GitHub Link -

click here (https://www.github.com/itsanil2011) to access project

Problem Statement

The ultimate goal of the project is to predict the views column, which represents the number of views a TED Talk is likely to receive.

General Guidelines : -

1. Well-structured, formatted, and commented code is required.
2. Exception Handling, Production Grade Code & Deployment Ready Code will be a plus. Those students will be awarded some additional credits.

   The additional credits will have advantages over other students during Star Student selection.

```
    [ Note: - Deployment Ready Code is defined as, the wh
ole .ipynb notebook should be executable in one go
            without a single error logged. ]
```

3. Each and every logic should have proper comments.
4. You may add as many number of charts you want. Make Sure for each and every chart the following format should be answered.

```
# Chart visualization code
```

- Why did you pick the specific chart?
- What is/are the insight(s) found from the chart?
- Will the gained insights help creating a positive business impact? Are there any insights that lead to negative growth? Justify with specific reason.

5. You have to create at least 15 logical & meaningful charts having important insights.

[Hints : - Do the Vizualization in a structured way while following "UBM" Rule.

U - Univariate Analysis,

B - Bivariate Analysis (Numerical - Categorical, Numerical - Numerical, Categorical - Categorical)

M - Multivariate Analysis]

6. You may add more ml algorithms for model creation. Make sure for each and every algorithm, the following format should be answered.

- Explain the ML Model used and it's performance using Evaluation metric Score Chart.
- Cross- Validation & Hyperparameter Tuning
- Have you seen any improvement? Note down the improvement with updates Evaluation metric Score Chart.
- Explain each evaluation metric's indication towards business and the business impact pf the ML model used.

Let's Begin !

1. Know Your Data

Import Libraries

In [10]:
```
# Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from wordcloud import WordCloud
from PIL import Image
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OrdinalEncoder
from sklearn.metrics import accuracy_score, classification_report
from sklearn.ensemble import StackingClassifier
from lightgbm import LGBMClassifier
from xgboost import XGBClassifier
from sentence_transformers import SentenceTransformer
from scipy.sparse import hstack, csr_matrix
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
import warnings; warnings.filterwarnings(action='ignore')
```

Dataset Loading

In [11]:
```
# Load Dataset
data_file = '/home/amruta/Downloads/Copy of data_ted_talks.csv'
d = pd.read_csv(data_file)
df = d.copy()
```

Dataset First View

In [12]:
```
# Dataset First Look
d.head()
```

Out[12]:

	talk_id	title	speaker_1	all_speakers	occupations	about_speakers	views	reco
0	1	Averting the climate crisis	Al Gore	{0: 'Al Gore'}	{0: ['climate advocate']}	{0: 'Nobel Laureate Al Gore focused the world'...	3523392	2
1	92	The best stats you've ever seen	Hans Rosling	{0: 'Hans Rosling'}	{0: ['global health expert; data visionary']}	{0: 'In Hans Rosling's hands, data sings. Glob...	14501685	2
2	7	Simplicity sells	David Pogue	{0: 'David Pogue'}	{0: ['technology columnist']}	{0: 'David Pogue is the personal technology co...	1920832	2
3	53	Greening the ghetto	Majora Carter	{0: 'Majora Carter'}	{0: ['activist for environmental justice']}	{0: 'Majora Carter redefined the field of envi...	2664069	2
4	66	Do schools kill creativity?	Sir Ken Robinson	{0: 'Sir Ken Robinson'}	{0: ['author', 'educator']}	{0: "Creativity expert Sir Ken Robinson challe...	65051954	2

Dataset Rows & Columns count

In [13]:
```
# Dataset Rows & Columns count
d.shape
```

Out[13]: `(4005, 19)`

Dataset Information

In [14]:
```
# Dataset Info
d.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4005 entries, 0 to 4004
Data columns (total 19 columns):
 #   Column          Non-Null Count  Dtype  
---  ------          --------------  -----  
 0   talk_id         4005 non-null   int64  
 1   title           4005 non-null   object 
 2   speaker_1       4005 non-null   object 
 3   all_speakers    4001 non-null   object 
 4   occupations     3483 non-null   object 
 5   about_speakers  3502 non-null   object 
 6   views           4005 non-null   int64  
 7   recorded_date   4004 non-null   object 
 8   published_date  4005 non-null   object 
 9   event           4005 non-null   object 
 10  native_lang     4005 non-null   object 
 11  available_lang  4005 non-null   object 
 12  comments        3350 non-null   float64
 13  duration        4005 non-null   int64  
 14  topics          4005 non-null   object 
 15  related_talks   4005 non-null   object 
 16  url             4005 non-null   object 
 17  description     4005 non-null   object 
 18  transcript      4005 non-null   object 
dtypes: float64(1), int64(3), object(15)
memory usage: 594.6+ KB
```

Duplicate Values

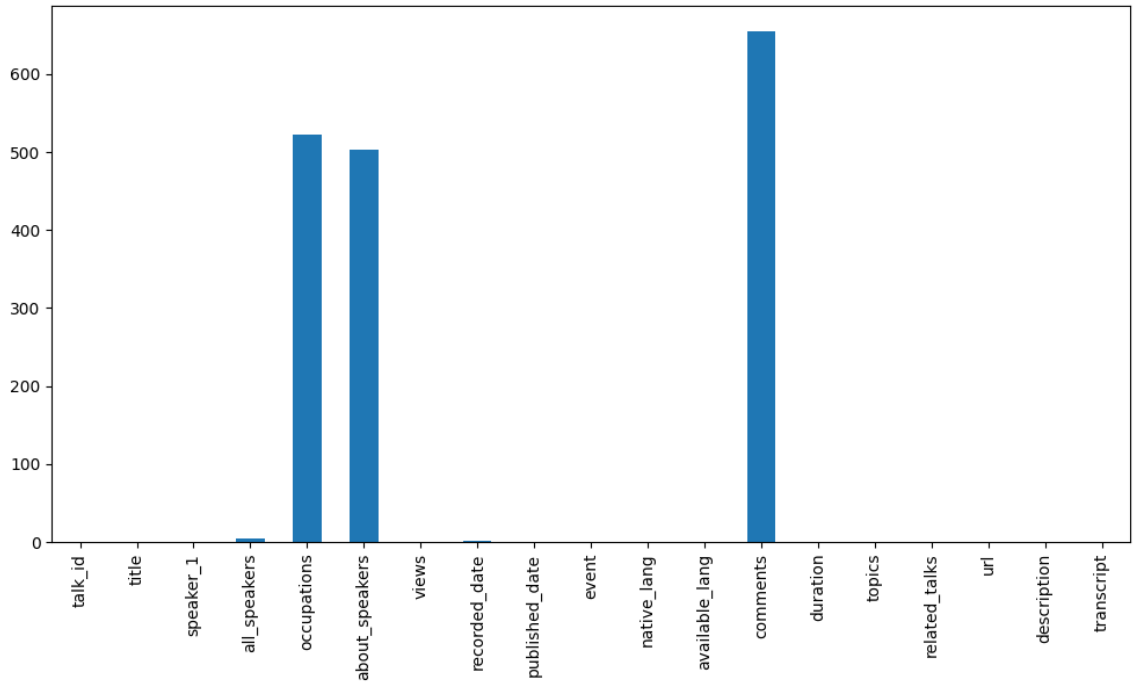
In [15]:
```
# Dataset Duplicate Value Count
df.drop_duplicates(inplace=True)
```

Missing Values/Null Values

In [16]:
```
# Missing Values/Null Values Count
df.isnull().sum().sort_values()
```

Out[16]:
```
talk_id             0
url                 0
related_talks       0
topics              0
duration            0
available_lang      0
native_lang         0
description         0
event               0
views               0
speaker_1           0
title               0
published_date      0
transcript          0
recorded_date       1
all_speakers        4
about_speakers    503
occupations       522
comments          655
dtype: int64
```

In [17]:
```
# Visualizing the missing values
plt.figure(figsize=(12,6))
df.isnull().sum().plot(kind='bar');
```



What did you know about your dataset?

Dataset having null values in following 3 columns

- **all_speakers:** 4
- **occupations:** 522

- **about_speakers:** 503
- **recorded_date:** 1
- **comments:** 655

In [18]:
```
# we have to fill null values from comment column with 0.
df.fillna({'comments': 0}, inplace=True)
```

In [19]:
```
# we have to fill null values from occupations column with missi
df.fillna({'occupations': 'missing'}, inplace=True)
```

In [20]:
```
# we have to fill null values from about_speakers column with mi
df.fillna({'about_speakers': 'missing'}, inplace=True)
```

In [21]:
```
df.views.value_counts()
```

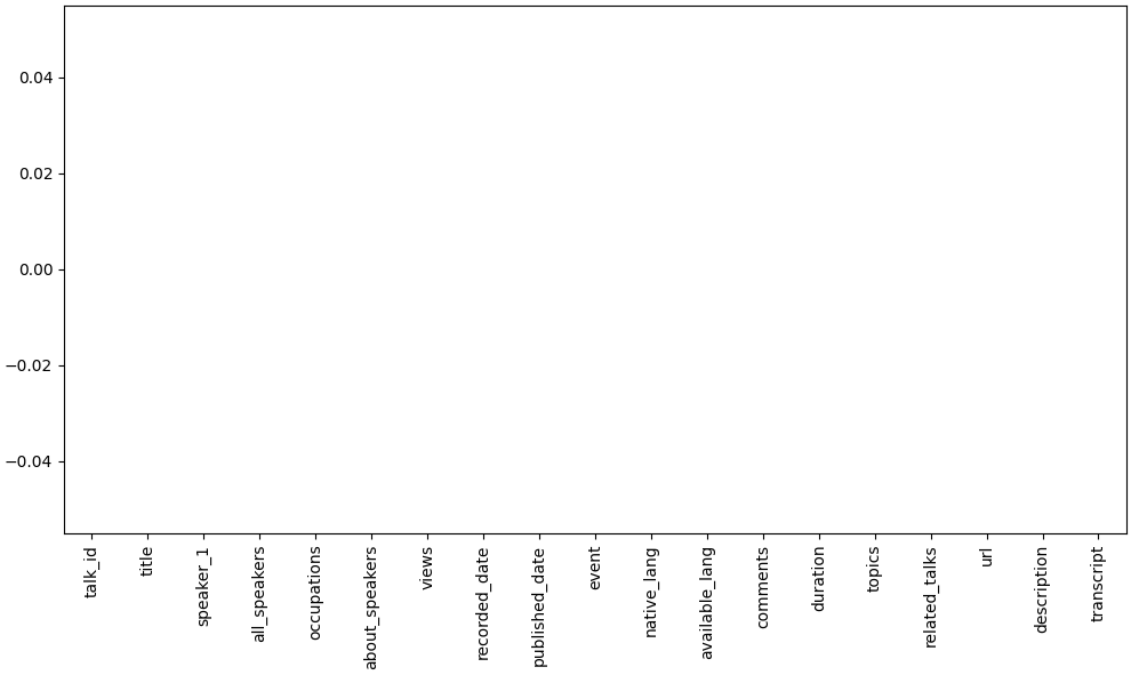
Out[21]:
```
views
0          6
1454261    2
1446787    2
1968363    2
806149     2
          ..
1152255    1
2625338    1
727048     1
1704167    1
56582      1
Name: count, Length: 3996, dtype: int64
```

There are 6 columns in views column having zero views. We simply replace it with zero.

In [22]:
```
df.views = df.views.replace(0, df.views.mean())
```

In [23]:
```
df.dropna(inplace=True)
```

In [24]:
```
# Visualizing the missing values
plt.figure(figsize=(12,6))
df.isnull().sum().plot(kind='bar');
```



Now no null values in dataframe

2. Understanding Your Variables

In [25]:
```
# Dataset Columns
df.columns
```

Out[25]:
```
Index(['talk_id', 'title', 'speaker_1', 'all_speakers', 'occupation
s',
       'about_speakers', 'views', 'recorded_date', 'published_date'
, 'event',
       'native_lang', 'available_lang', 'comments', 'duration', 'to
pics',
       'related_talks', 'url', 'description', 'transcript'],
      dtype='object')
```

In [26]:
```
# Dataset Describe
df.describe()
```

Out[26]:

	talk_id	views	comments	duration
count	4000.000000	4.000000e+03	4000.00000	4000.000000
mean	12397.454500	2.152523e+06	135.66750	723.959250
std	17425.490867	3.451848e+06	253.18078	361.788987
min	1.000000	1.000400e+04	0.00000	60.000000
25%	1250.750000	8.857095e+05	17.00000	392.750000
50%	2330.500000	1.379386e+06	68.00000	738.000000
75%	23770.500000	2.143401e+06	162.25000	973.000000
max	62794.000000	6.505195e+07	6449.00000	3922.000000

Variables Description

columns in the dataset are as follows:

- **talk_id:** A unique identifier for each TED Talk.
- **title:** The title of the TED Talk, which often reflects the main theme of the talk.
- **speaker_1:** The name of the main speaker of the TED Talk.
- **all_speakers:** The names of all speakers who participated in the talk.
- **occupations:** The professions of the speakers.
- **about_speakers:** A brief description of the speaker(s), providing context about their background.
- **views:** The total number of views the TED Talk has received. This is the target variable for the prediction model.
- **recorded_date:** The date on which the talk was recorded.
- **published_date:** The date on which the talk was made available online.
- **event:** The event or conference where the talk was presented.
- **native_lang:** The native language(s) of the speaker(s).
- **available_lang:** The language(s) in which the talk is available.
- **comments:** The number of comments the TED Talk has received.
- **duration:** The length of the TED Talk, typically measured in minutes.
- **topics:** A list of topics covered in the TED Talk.
- **related_talks:** Other TED Talks related to the current one, based on theme or subject matter.
- **url:** A direct URL to the TED Talk.
- **description:** A brief description or summary of the TED Talk.
- **transcript:** The full transcript of the TED Talk.

Check Unique Values for each variable.

In [27]:
```
# Check Unique Values for each variable.
df.nunique()
```

Out[27]:
```
talk_id           4000
title             4000
speaker_1         3269
all_speakers      3305
occupations       2050
about_speakers    2977
views             3991
recorded_date     1333
published_date    2962
event              458
native_lang         12
available_lang    3898
comments           601
duration          1188
topics            3972
related_talks     4000
url               4000
description       4000
transcript        4000
dtype: int64
```

3. *Data Wrangling*

Data Wrangling Code

In [28]:
```
# Write your code to make your dataset analysis ready.
df.all_speakers.sample(4)
```

Out[28]:
```
2714      {0: 'Eve Abrams'}
429      {0: 'Sarah Jones'}
2971    {0: 'Kim Preshoff'}
3062     {0: 'Jac de Haan'}
Name: all_speakers, dtype: object
```

converted values of column all_speakers from dict to str

In [29]:
```
df['all_speakers'] = df.all_speakers.apply( lambda x: list(eval(x
```

In [30]:
```
1 df[['speaker_1','all_speakers']].sample(12)
```

Out[30]:

	speaker_1	all_speakers
3468	Doug Roble	[Doug Roble]
2196	Lisa Dyson	[Lisa Dyson]
3470	Carson Bruns	[Carson Bruns]
2128	Hugh Evans	[Hugh Evans]
1059	Lauren Hodge	[Lauren Hodge, Shree Bose, Naomi Shah]
1857	Ben Ambridge	[Ben Ambridge]
2326	Jeff Speck	[Jeff Speck]
3241	Dolores Huerta	[Dolores Huerta]
1712	Will Potter	[Will Potter]
1606	Mark Kendall	[Mark Kendall]
2266	Halla Tómasdóttir	[Halla Tómasdóttir]
3407	Jean Baptiste Koehl	[Jean Baptiste Koehl]

by executing above line of code sevral times we can conclude that *speaker1* and *all speakers* having exactly same value. So simply we will delete all_speakers column.

In [31]:
```
1 df.drop(columns=['all_speakers'], inplace=True)
```

In [32]:
```
1 df.occupations.sample(5)
```

Out[32]:
```
342                    {0: ['inventor']}
3614             {0: ['soil scientist']}
2201    {0: ['machine learning expert']}
1676           {0: ['plasma physicist']}
448               {0: ['entrepreneur']}
Name: occupations, dtype: object
```

occupations column having messy data we have to clean it by converting values into plain text

In [33]:
```
1 df.occupations = df.occupations.apply( lambda name: eval(name).ge
```

In [34]:
```
1 df.occupations.sample(5)
```

Out[34]:
```
177                 college president
2339                         designer
2762          cultural media builder
2420    citizen biomedical researcher
1376             food security expert
Name: occupations, dtype: object
```

In [35]:
```
1 df.about_speakers.sample(5)
```

Out[35]:
```
956     {0: 'Lucianne Walkowicz works on NASA\'s Keple...
805     {0: 'Thomas Thwaites is a designer "of a more ...
3785    {0: 'Jasmine Crowe is the creator of Goodr, a ...
3568    {0: 'Elizabeth Howell is working to address ma...
3083                                               missing
Name: about_speakers, dtype: object
```

about_speakers column having messy data we have to clean it by converting values into plain text

In [36]:
```
1 df.about_speakers = df.about_speakers.apply( lambda name: eval(na
```

In [37]:
```
1 df.about_speakers.sample(5)
```

Out[37]:
```
2351    Carrie Nugent is part of a team that uses NASA...
3612                                               missing
2686    A theoretical physicist by training, IBM's Sim...
1942    Donald Hoffman studies how our visual percepti...
941     Markus Fischer led the team at Festo that deve...
Name: about_speakers, dtype: object
```

converting recorded_date and published_date column into datetime format

In [38]:
```
1 df['recorded_date'] = pd.to_datetime(df['recorded_date'])
2 df['published_date'] = pd.to_datetime(df['published_date'])
```

In [39]:
```
1 df.available_lang.sample(5)
```

Out[39]:
```
2366    ['ar', 'en', 'es', 'fr', 'hu', 'it', 'ko', 'pl...
708     ['am', 'ar', 'bg', 'bn', 'cs', 'de', 'el', 'en...
1145    ['ar', 'bg', 'cs', 'da', 'de', 'el', 'en', 'es...
2358    ['ar', 'da', 'el', 'en', 'es', 'fr', 'hu', 'it...
3384    ['ar', 'de', 'en', 'es', 'fr', 'he', 'id', 'it...
Name: available_lang, dtype: object
```

avaliable_lang column having language codes. We can convert this column to store language count

In [40]:
```
1 df.available_lang = df.available_lang.apply(lambda x: len(eval(x)
```

In [42]:
```
1 df.available_lang.sample(3)
```

Out[42]:
```
2626    18
49      22
1486    31
Name: available_lang, dtype: int64
```

All manipulations

- converted values of column all_speakers from dict to str.
- converted occupations column values into plain text.
- converted about_speakers into plain text.
- converted recorded_date column into datetime format.
- converted recorded_date column into datetime format.

---

4. Data Vizualization, Storytelling & Experimenting with charts : Understand the relationships between variables

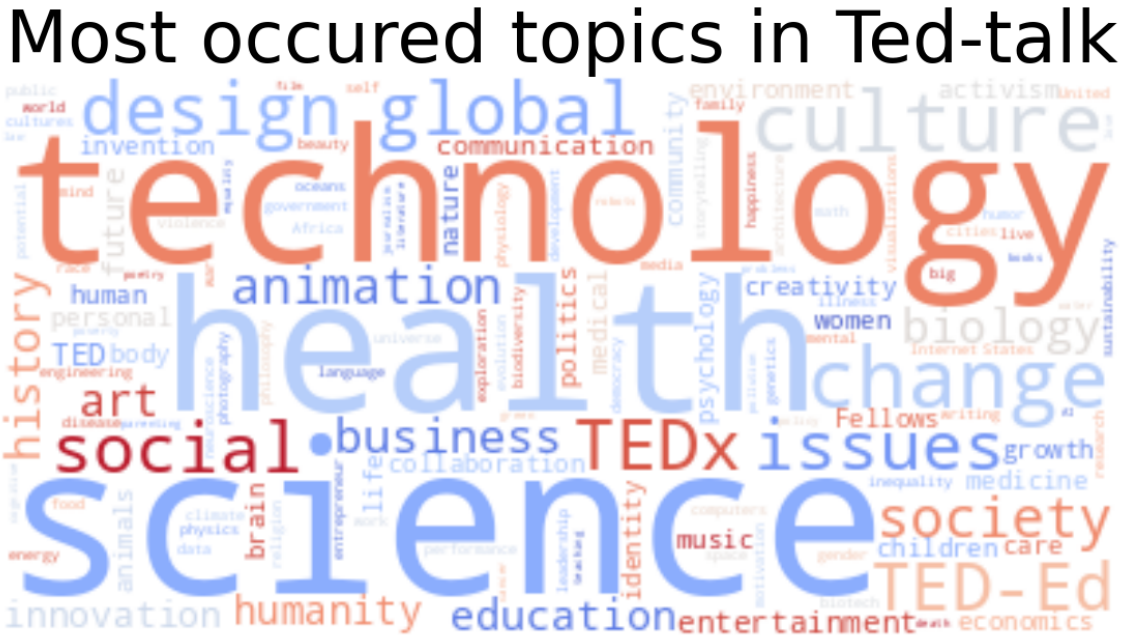
Chart - 1

In [46]:

```
# for topics column
words = []
for i in df.topics.values:
  for j in i.split():
    words.append(j.replace('[', '').replace(']', '').replace("'",

wc = pd.Series(words).value_counts()

# Generate the word cloud with the custom mask
wordcloud = WordCloud(background_color='white',colormap='coolwarm
plt.figure(figsize=(14, 25))
plt.imshow(wordcloud, interpolation='bilinear')
plt.title('Most occured topics in Ted-talk', fontdict={'size': 50
plt.axis('off');
```



1. Why did you pick the specific chart?

- topics column having all string values.
- and these string values are individual.
- So, converting these values using column encoding will not work.
- Thats why WordCloud is best chart for this data.

2. What is/are the insight(s) found from the chart?

following are the most occured words in Topics column

- technology
- science
- health
- change
- global

3. Will the gained insights help creating a positive business impact?

Are there any insights that lead to negative growth? Justify with specific reason.

Following are the less occured words in topics:

- humanity
- creativity
- medicine
- children
- innovation

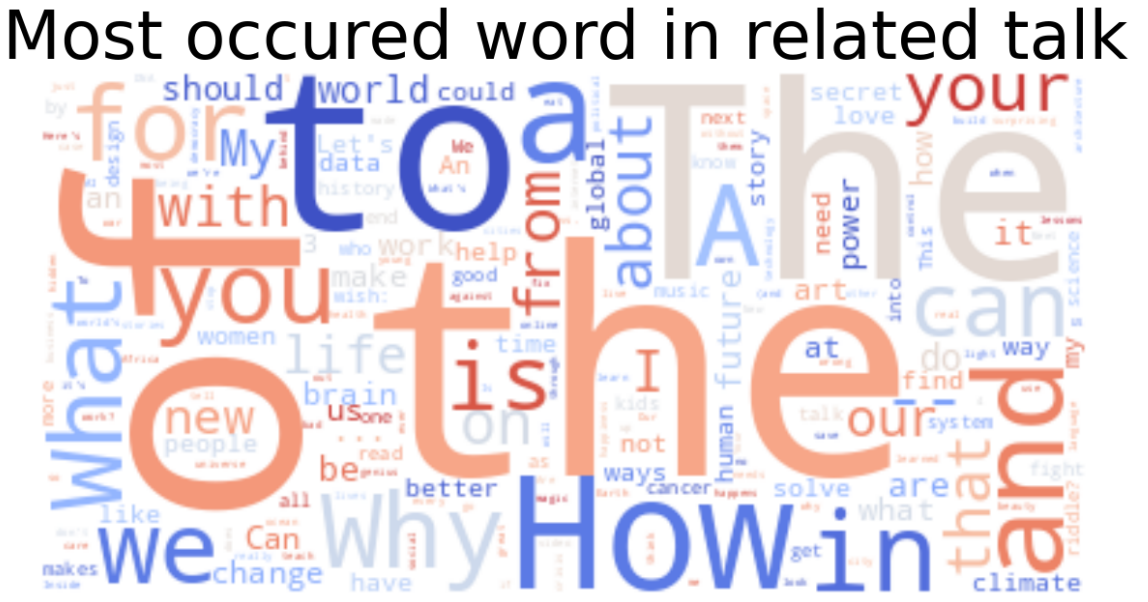
In my view these topics are most important and should be included in discussion.

Chart - 2

In [47]:
```
# Chart - 2 visualization code
words = []
for value in df.related_talks.values:
  for i in eval(value).values():
    for j in i.split():
      words.append(j)

wc = pd.Series(words).value_counts()

# Generate the word cloud with the custom mask
wordcloud = WordCloud(background_color='white',colormap='coolwarm
plt.figure(figsize=(14, 25))
plt.imshow(wordcloud, interpolation='bilinear')
plt.title('Most occured word in related talk', fontdict={'size':
plt.axis('off');
```



1. Why did you pick the specific chart?

- related talk column having all string values.

- and these string values are individual.
- So, converting these values using column encoding will not work.
- Thats why WordCloud is best chart for this data.

2. What is/are the insight(s) found from the chart?

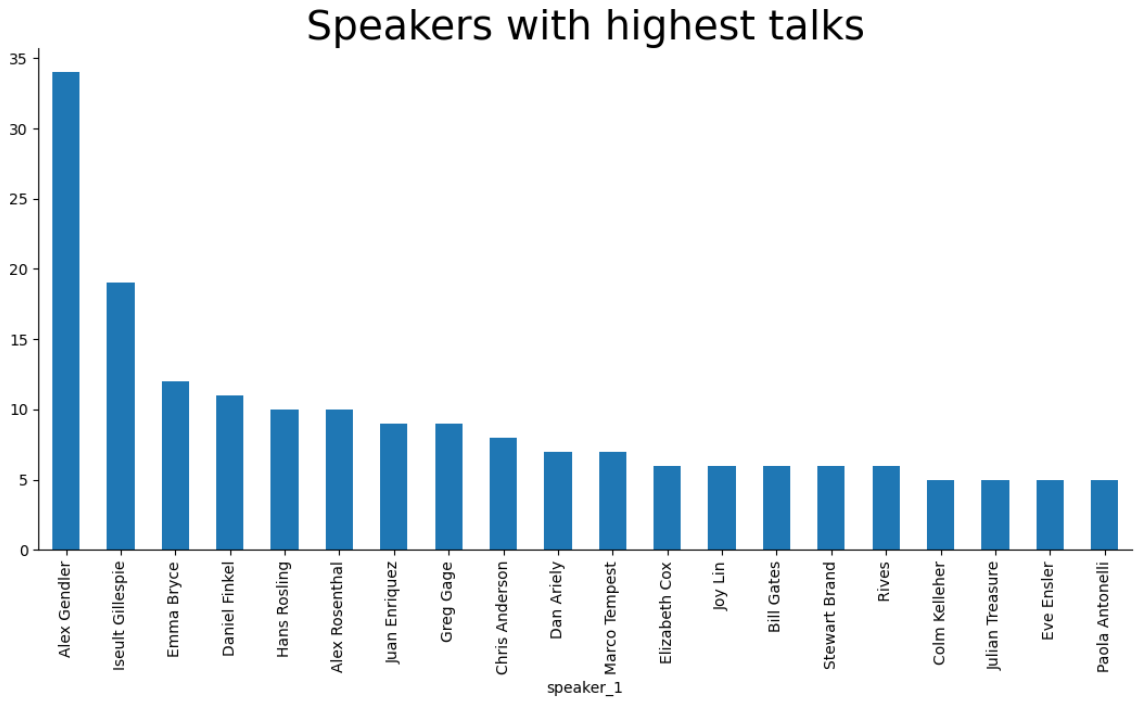
- of, the, to, how, for these are the most occured words.

3. Will the gained insights help creating a positive business impact?

- No

Chart - 3

In [48]:
```
# Chart - 3 visualization code
plt.figure(figsize=(13,6))
plt.gca().spines['top'].set_visible(False)
plt.gca().spines['right'].set_visible(False)
plt.title('Speakers with highest talks', fontdict={'size': 27})
df.speaker_1.value_counts()[:20].plot(kind='bar') ;
```



1. Why did you pick the specific chart?

- I was trying to view speakers with highest count.
- For this bar chart is perfect option.

2. What is/are the insight(s) found from the chart?

Following are the top 5 speakers, in which 3 speakers are from academics/authors.

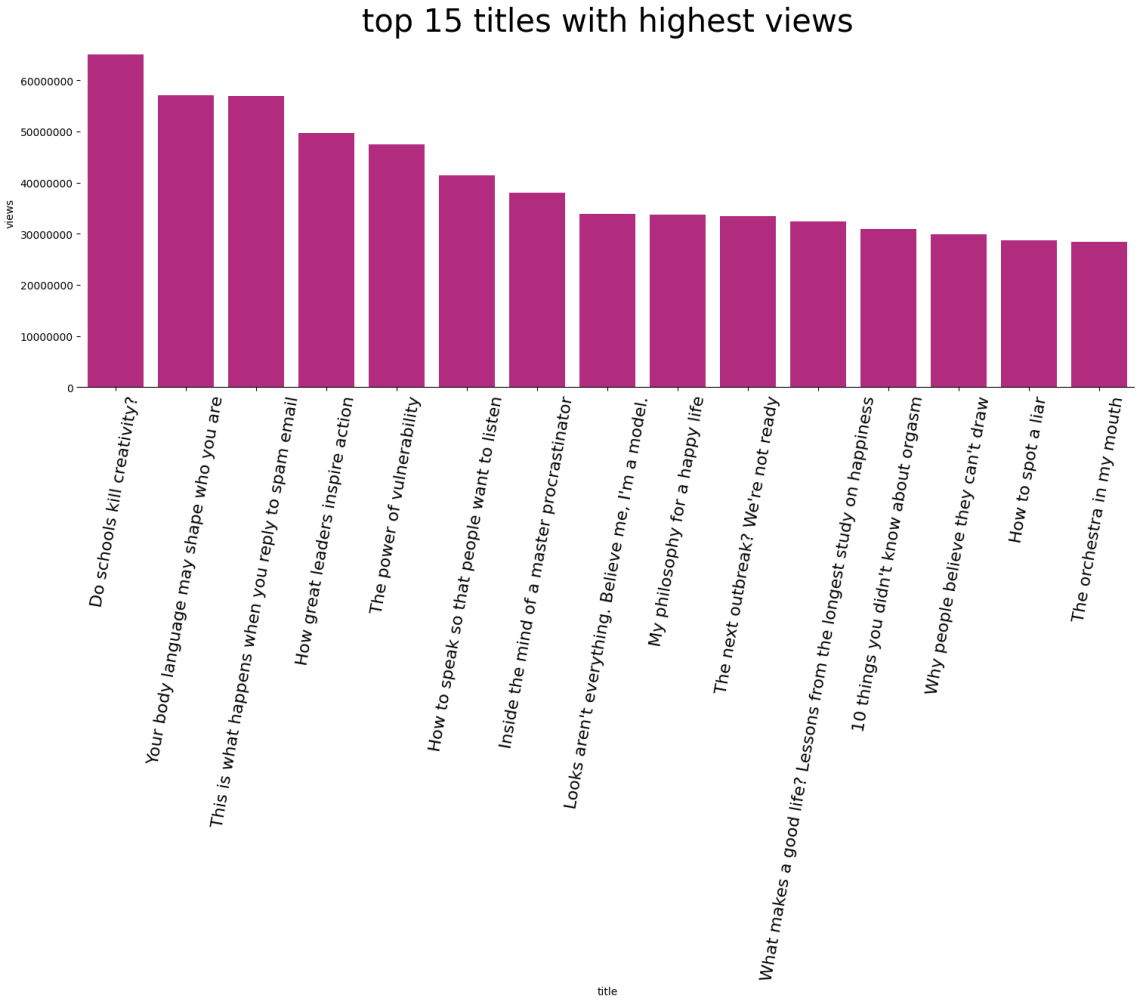
1. Hans Rosling - Swedish physician and academic
2. Juan Enríquez - Mexican author
3. Greg Gage - Neuroscientist & Researcher
4. Dan Ariely - American professor and author
5. Marco Tempest - Swiss magician

Chart - 4

In [53]:
```
# Chart - 4 visualization code
from matplotlib.ticker import ScalarFormatter
top_15_titles = df.sort_values(by='views', ascending=False).head(
plt.figure(figsize=(18,6))

plt.gca().yaxis.set_major_formatter(ScalarFormatter(useMathText=T
plt.ticklabel_format(style='plain', axis='y')

plt.gca().spines['top'].set_visible(False)
plt.gca().spines['right'].set_visible(False)
plt.gca().spines['left'].set_visible(False)
plt.xticks(rotation=80, fontsize=16)
plt.title('top 15 titles with highest views', fontdict={'size': 3
sns.barplot(x='title', y= 'views', data=top_15_titles, color='#C1
```



1. Why did you pick the specific chart?

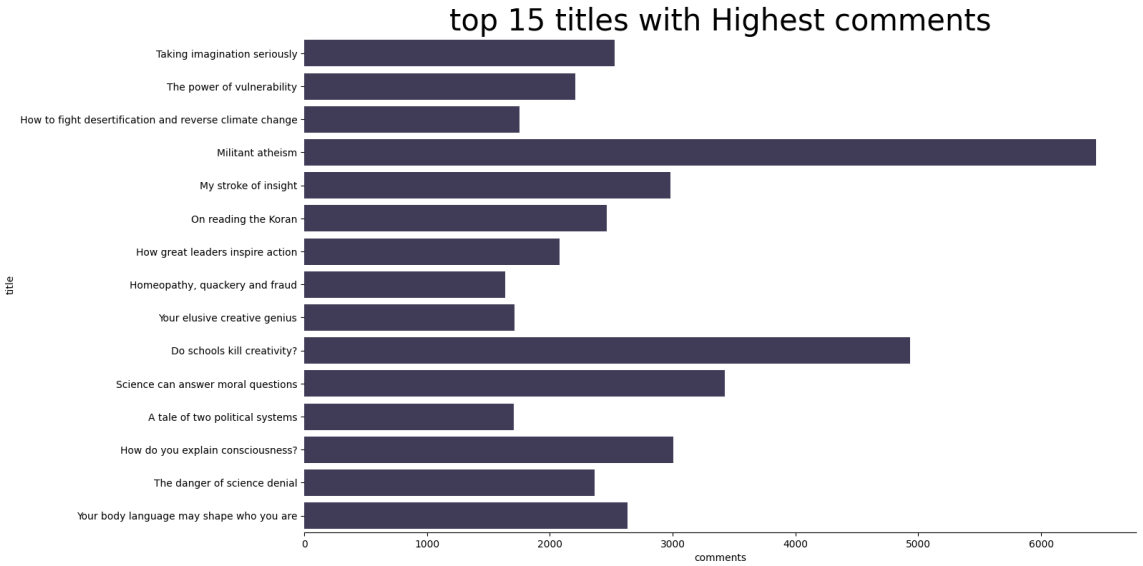
- To visualize view count of the top 15 titles, bar chart is the best option.

2. What is/are the insight(s) found from the chart?

- Do schools kill creativity is the really good topic viewed more in talks.
- Second most viewed topic is about spam mails, it is useful in todays technological world.

Chart - 5

In [61]:
```
1
2 top_10_titles = df.sort_values(by='comments', ascending=False).he
3 plt.figure(figsize=(15,9))
4 plt.gca().spines['top'].set_visible(False)
5 plt.gca().spines['right'].set_visible(False)
6 plt.gca().spines['left'].set_visible(False)
7
8 plt.title('top 15 titles with Highest comments', fontdict={'size
9 sns.barplot(y='title', x= 'comments', data=top_10_titles, color=
```



1. Why did you pick the specific chart?

- To visualize data like frequency count, bar chart is good option.

2. What is/are the insight(s) found from the chart?

- militant atheism
- Do school kill creativity?
- Science can answer moral questions

Are the top title with highest comments.

3. Will the gained insights help creating a positive business impact?

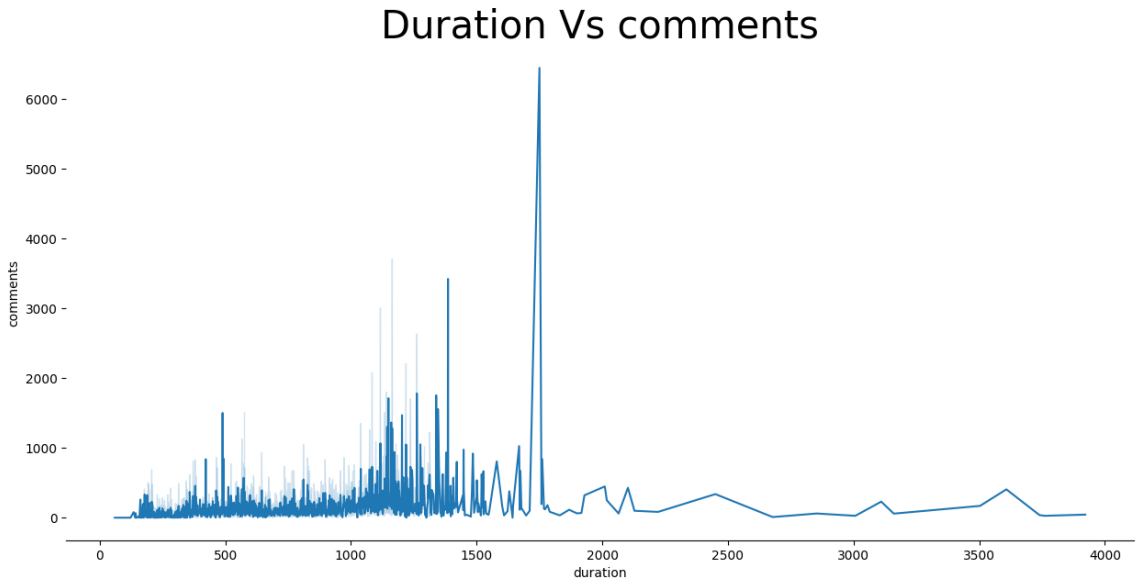
Are there any insights that lead to negative growth? Justify with specific reason.

- A tale of two political system having least comments.
- it means peoples are less interested in politics.

Chart - 6

In [62]:
```
plt.figure(figsize=(15,7))
plt.gca().spines['top'].set_visible(False)
plt.gca().spines['right'].set_visible(False)
plt.gca().spines['left'].set_visible(False)

plt.title('Duration Vs comments', fontdict={'size': 30})
sns.lineplot(x='duration', y='comments', data=df);
```



1. Why did you pick the specific chart?

- Both the values from Duration column and Comment columns are nemerical. Thats why this chart is selected.

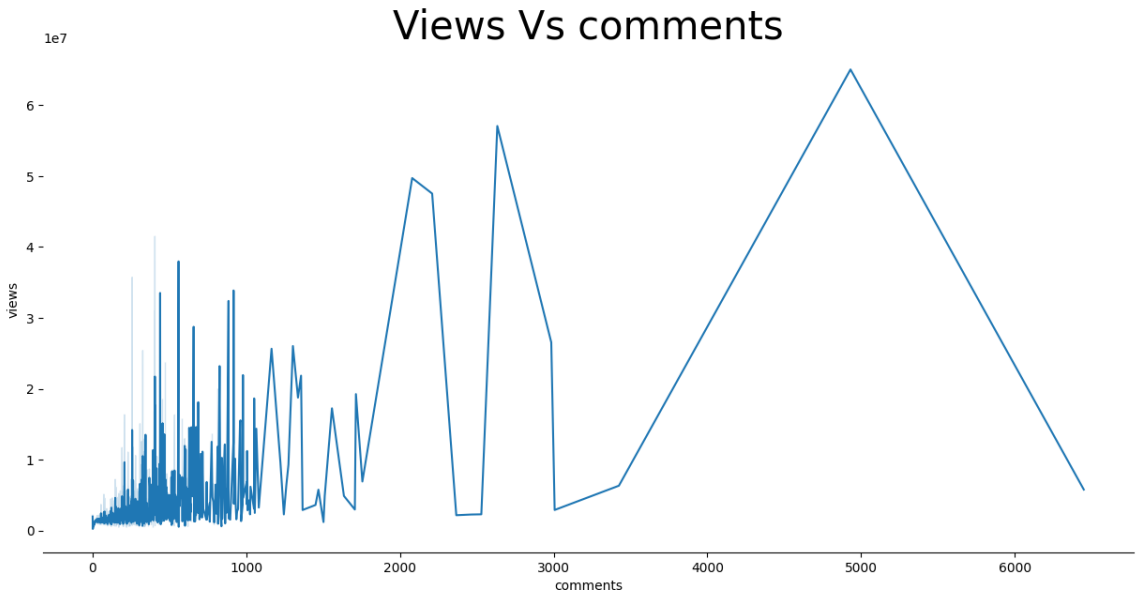
2. What is/are the insight(s) found from the chart?

- Comments density is higher when duration is between around 200 and 1500.
- comments density is lower when duration is greater than around 1800.

Chart - 7

In [63]:
```
plt.figure(figsize=(15,7))
plt.gca().spines['top'].set_visible(False)
plt.gca().spines['right'].set_visible(False)
plt.gca().spines['left'].set_visible(False)

plt.title('Views Vs comments', fontdict={'size': 30})
sns.lineplot(x='comments', y='views', data=df);
```



1. Why did you pick the specific chart?

- Both the columns having linear relation thats why picked this chart.

2. What is/are the insight(s) found from the chart?

High Variability in Comments for Lower View Counts

- In the lower range of views (0 - 10 million), the number of comments fluctuates significantly. Some videos with similar views have vastly different comment counts, suggesting that engagement varies based on content type or audience interest.

Outliers with Unusually High Comments

- There are a few extreme spikes where the number of comments is disproportionately high compared to surrounding data points. This might indicate viral content, controversial topics, or highly engaging discussions.

Steady Increase in Comments for Higher Views

- As the view count increases beyond 10 million, the comment count generally trends upward, indicating that higher viewership typically leads to more engagement. However, there are still fluctuations, meaning not all high-view videos receive consistent engagement.

Sudden Drop-Offs and Gaps in Data

- Some sections of the graph show abrupt drops in comments despite increasing views. This could be due to platform-specific factors like disabled comments, less engaging content, or differences in audience behavior.

3. Will the gained insights help creating a positive business impact?

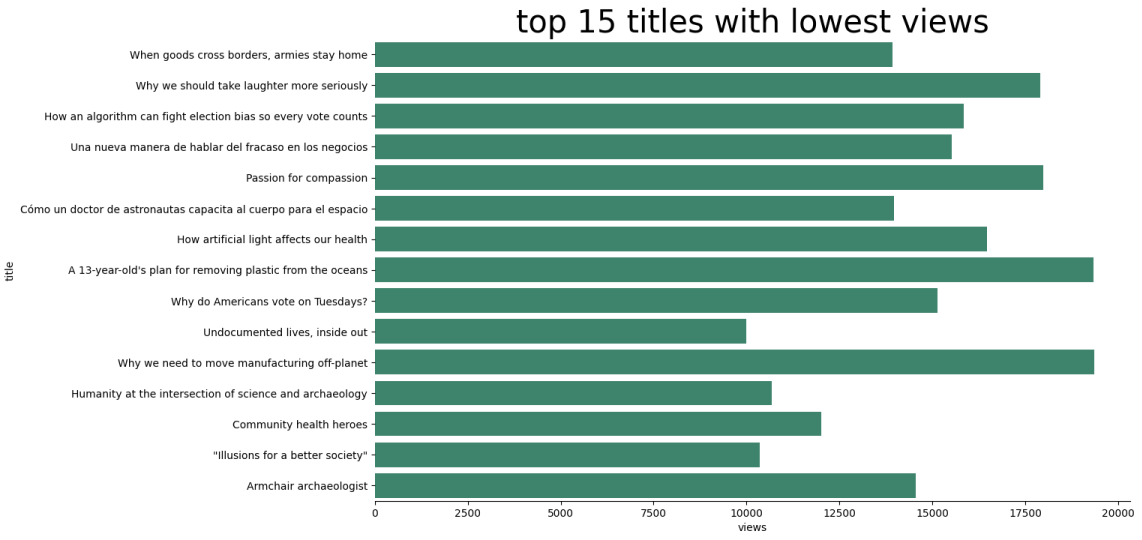
Yes, the insights from this graph can help drive positive business impact in several ways, particularly in content strategy, audience engagement, and monetization. Here's how:

1. Optimizing Content Strategy
2. Enhancing Audience Engagement
3. Improving Monetization Strategies
4. Predicting Trends & Reducing Inefficiencies

Chart - 8

In [68]:
```
top_15_titles = df.sort_values(by='views', ascending=True).head(1
plt.figure(figsize=(13,8))
plt.gca().spines['top'].set_visible(False)
plt.gca().spines['right'].set_visible(False)
plt.gca().spines['left'].set_visible(False)

plt.title('top 15 titles with lowest views', fontdict={'size': 30
sns.barplot(y='title', x= 'views', data=top_15_titles, color='#32
```

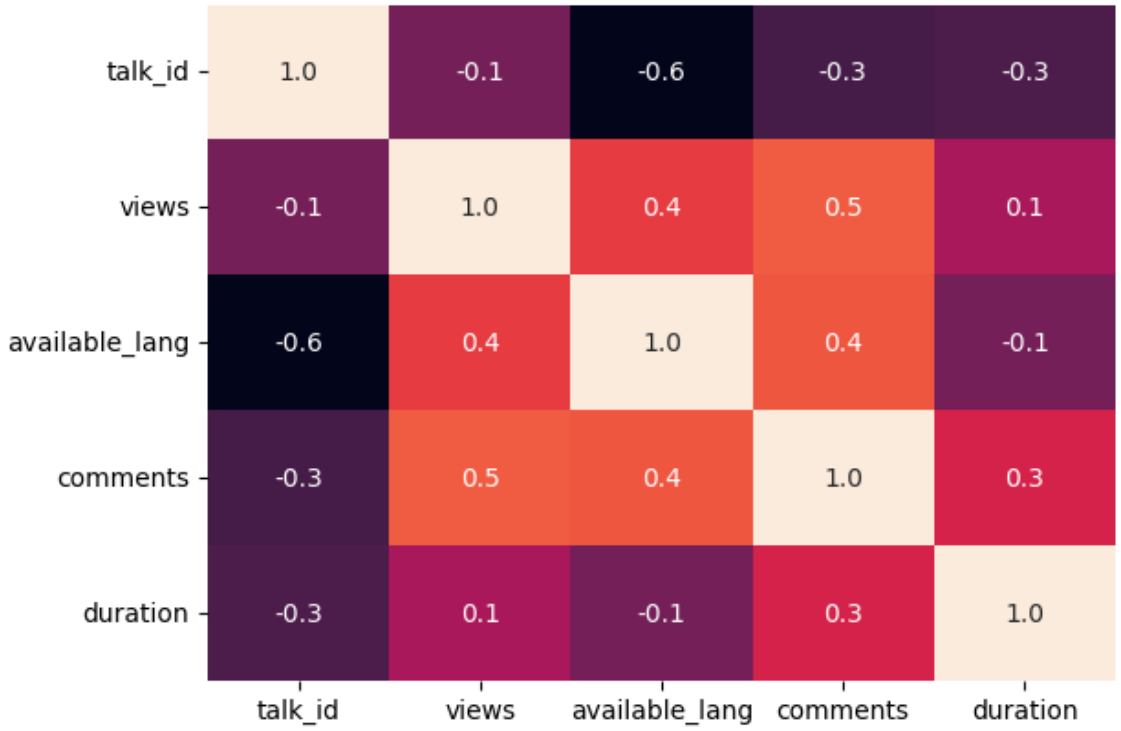


1. Why did you pick the specific chart?

- To visualize top 15 title with lowest view bar chart is good.

Chart - 14 - Correlation Heatmap

In [69]:
```
# Correlation Heatmap visualization code
sns.heatmap(df.corr(numeric_only=True), annot=True, fmt='.1f', cb
```

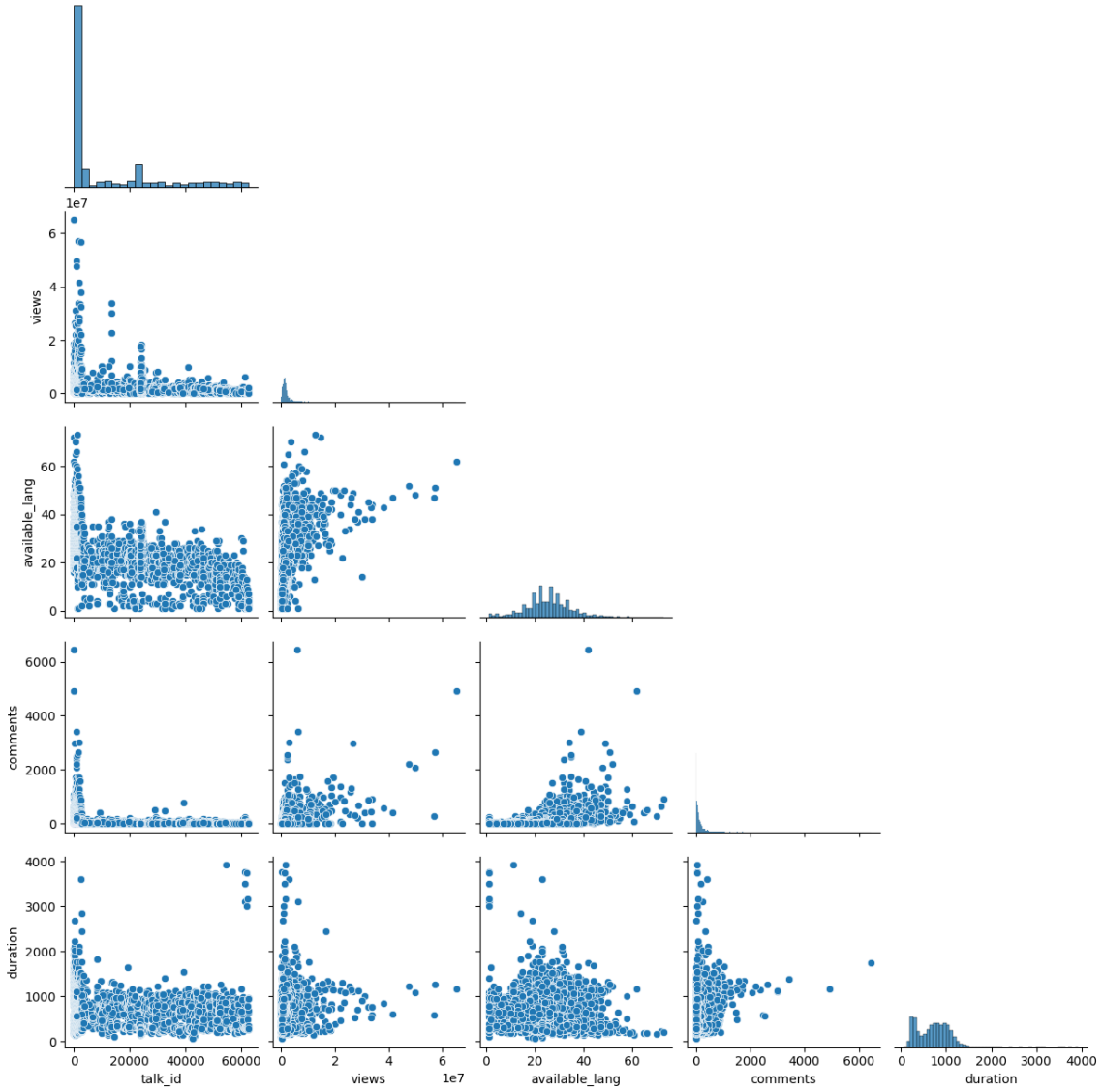


1. Why did you pick the specific chart?

- heatmap is widely used to see correlation.

Chart - 15 - Pair Plot

In [70]:
```
# Pair Plot visualization code
sns.pairplot(df,corner=True);
```



1. Why did you pick the specific chart?

- Pairplot used to see relation between different columns of dataframe.

Data preprocessing

drop unnecessary columns

In [71]:
```
df.drop(columns=['talk_id', 'transcript'], inplace=True)
```

Define binary classes: Low = 0, High = 1

In [72]:
```
q_low = df['views'].quantile(0.33)
q_high = df['views'].quantile(0.66)
df_binary = df[(df['views'] <= q_low) | (df['views'] > q_high)].
df_binary['popularity'] = (df_binary['views'] > q_high).astype(i
```

Feature engineering

In [73]:
```
df_binary['publish_delay_days']    = (df_binary['published_date'
df_binary['description_length']    = df_binary['description'].st
df_binary['title_length']          = df_binary['title'].str.len(
df_binary['about_speakers_length'] = df_binary['about_speakers']
df_binary['num_topics']            = df_binary['topics'].apply(la
df_binary['duration_x_description'] = df_binary['duration'] * df_
df_binary['topics_x_title']         = df_binary['num_topics'] * c
```

Encode categorical features

In [74]:
```
cat_features = ['speaker_1','occupations','event','native_lang']
enc = OrdinalEncoder(handle_unknown='use_encoded_value', unknown_
df_binary[cat_features] = enc.fit_transform(df_binary[cat_feature
```

Numerical features

In [75]:
```
num_feats_binary = df_binary[[
    'comments','duration','speaker_1','occupations','event','nati
    'publish_delay_days','description_length','title_length',
    'about_speakers_length','num_topics','duration_x_description
]].values
```

Sentence embeddings

In [76]:
```
model_name = 'all-MiniLM-L6-v2'
st_model = SentenceTransformer(model_name)
desc_emb_binary    = st_model.encode(df_binary['description'].to
title_emb_binary   = st_model.encode(df_binary['title'].tolist()
speaker_emb_binary = st_model.encode(df_binary['about_speakers']
```

modules.json: 100% 349/349 [00:00<00:00, 7.60kB/s]

config_sentence_transformers.json: 100% 116/116 [00:00<00:00, 4.13kB/s]

README.md: 100% 10.5k/10.5k [00:00<00:00, 389kB/s]

sentence_bert_config.json: 100% 53.0/53.0 [00:00<00:00, 1.98kB/s]

config.json: 100% 612/612 [00:00<00:00, 21.5kB/s]

```
Xet Storage is enabled for this repo, but the 'hf_xet' package is n
ot installed. Falling back to regular HTTP download. For better per
formance, install the package with: `pip install huggingface_hub[hf
_xet]` or `pip install hf_xet`
```

model.safetensors: 100% 90.9M/90.9M [00:37<00:00, 2.14MB/s]

tokenizer_config.json: 100% 350/350 [00:00<00:00, 12.7kB/s]

vocab.txt: 100% 232k/232k [00:00<00:00, 2.54MB/s]

tokenizer.json: 100% 466k/466k [00:00<00:00, 1.12MB/s]

special_tokens_map.json: 100% 112/112 [00:00<00:00, 1.46kB/s]

config.json: 100% 190/190 [00:00<00:00, 3.95kB/s]

```
Batches: 100%                                                    84/84 [00:57<00:00,  3.18it/
s]

Batches: 100%                                                    84/84 [00:10<00:00, 12.49it/
s]

Batches: 100%                                                    84/84 [00:28<00:00, 19.10it/
s]
```

Combine features

In [77]:
```
X = hstack([
    csr_matrix(num_feats_binary),
    csr_matrix(desc_emb_binary),
    csr_matrix(title_emb_binary),
    csr_matrix(speaker_emb_binary)
])
y = df_binary['popularity'].values

```

Train/test split

In [78]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

Model: LightGBM (or stacked)

In [79]:
```
model = LGBMClassifier(n_estimators=300, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

```
[LightGBM] [Info] Number of positive: 1088, number of negative: 105
6
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overh
ead of testing was 0.073960 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 296379
[LightGBM] [Info] Number of data points in the train set: 2144, num
ber of used features: 1165
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.507463 -> initsco
re=0.029853
[LightGBM] [Info] Start training from score 0.029853
```

Evaluation

In [80]:
```
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test,
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

```
Accuracy: 0.8264925373134329

Classification Report:
               precision    recall  f1-score   support

          Low       0.82      0.83      0.82       264
         High       0.83      0.82      0.83       272

     accuracy                           0.83       536
    macro avg       0.83      0.83      0.83       536
 weighted avg       0.83      0.83      0.83       536


Confusion Matrix:
 [[219  45]
 [ 48 224]]
```

Logistic Regression for TED Talk Popularity (Binary)

Prepare features and target from earlier steps

In [81]:
```
X = hstack([
    csr_matrix(num_feats_binary),
    csr_matrix(desc_emb_binary),
    csr_matrix(title_emb_binary),
    csr_matrix(speaker_emb_binary)
])
y = df_binary['popularity'].values
```

Train/Test Split

In [82]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)
```

Logistic Regression Model

In [84]:
```
lr_model = LogisticRegression(max_iter=1000, solver='lbfgs', )
lr_model.fit(X_train, y_train)
```

Out[84]:
```
▾ LogisticRegression
LogisticRegression(max_iter=1000)
```

Predict and Evaluate

In [85]:
```
y_pred = lr_model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test,
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))

```

```
Accuracy: 0.710820895522388

Classification Report:
               precision    recall  f1-score   support

          Low       0.68      0.80      0.73       264
         High       0.76      0.63      0.69       272

     accuracy                           0.71       536
    macro avg       0.72      0.71      0.71       536
 weighted avg       0.72      0.71      0.71       536


Confusion Matrix:
 [[210  54]
 [101 171]]
```

E N D